Лекция 5: Принципы SOLID (Часть 2). LSP, ISP, DIP. Внедрение зависимостей (DI) и Midterm

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Аудитория: Бакалавриат, 2-й курс, специальность «6B06110 Программная инженерия»
Формат: Лекция, 2 академических часа (90 минут)
Методологический нарратив: Order (Завершение фундамента SOLID. Архитектурная инверсия)

Введение и инженерный контекст

На прошлой неделе мы изучили первые два принципа SOLID: SRP (ответственность перед одним актором) и OCP (расширение через полиморфизм без изменения существующего кода).
Сегодня мы завершаем освоение акронима SOLID тремя принципами, которые определяют правила построения интерфейсов, иерархий типов и связывания компонентов системы: - **L** — Liskov Substitution Principle (Принцип подстановки Барбары Лисков); - **I** — Interface Segregation Principle (Принцип разделения интерфейсов); - **D** — Dependency Inversion Principle (Принцип инверсии зависимостей).
Особое внимание мы уделим паттерну **Dependency Injection (Внедрение зависимостей)** — главному механизму борьбы с жесткой связностью в промышленной разработке. Также мы подведем итоги первой половины семестра перед рубежным контролем (Midterm).

План лекции (90 минут)

№	Время	Тема	Формат
1	00:00–00:25	Liskov Substitution Principle (LSP): контрактное проектирование (Design by Contract), предусловия и постусловия	Архитектурная теория
2	00:25–00:45	Interface Segregation Principle (ISP): вред «толстых» интерфейсов, ролевые интерфейсы	Разбор антипаттернов
3	00:45–01:10	Dependency Inversion Principle	Теория + код

№	Время	Тема	Формат
		(DIP): инверсия контроля (IoC), Dependency Injection (DI)	
4	01:10–01:25	DI-контейнеры в Python: связывание графа объектов без жестких зависимостей	Практическая демонстрация
5	01:25–01:30	Подготовка к Midterm и рубежному контролю РК1	Рефлексия

---

1. Liskov Substitution Principle (LSP)

Сформулирован Барбарой Лисков в 1987 году: > **«Если для каждого объекта o_1 типа S существует объект o_2 типа T такой, что для всех программ P, определенных в терминах T, поведение P не изменяется при подстановке o_1 вместо o_2, то S является подтипом T».**
Человеческим языком: **Подкласс не должен нарушать ожидания клиентского кода, работающего с базовым классом**. Если функция ожидает базовый тип, она должна корректно работать с любым его потомком без специальных проверок и сюрпризов.

Правила контракта (Design by Contract):

1. **Предусловия не могут быть усилены в подклассе**: подкласс не может требовать более жестких условий на входе, чем требовал базовый класс.
2. **Постусловия не могут быть ослаблены в подклассе**: подкласс должен гарантировать как минимум тот же результат на выходе, что и базовый класс.
3. **Инварианты базового класса должны сохраняться**: подкласс не имеет права переводить объект в недопустимое состояние.
4. **Правило истории**: подкласс не должен изменять состояние, которое базовый класс декларировал как неизменяемое.

Нарушение LSP в транзитной системе:

```
# Базовый контракт
class TransitCard:
    def charge(self, amount: Decimal) -> None:
        """Списывает деньги. Выбрасывает InsufficientFundsError при нехватке."""
        ...

# Нарушение LSP: Виртуальный бесплатный проездной
class FreeVolunteerPass(TransitCard):
    def charge(self, amount: Decimal) -> None:
        # ОШИБКА: подкласс выбрасывает неожиданное исключение OperationNotSupported
Error,
```

```
        # ломая весь процессинг, который ожидал нормального поведения TransitCard!
        raise NotImplementedError("С волонтерского билета нельзя списывать средства!")
```

Клиентский код, написанный под TransitCard, рухнет при попытке вызвать charge().
Решение: разделять иерархию на интерфейс верификации проезда (ValidatablePass) и интерфейс финансового списания (ChargeableCard).

---

2. Interface Segregation Principle (ISP)

Сформулирован Робертом Мартином: > **«Клиенты не должны зависеть от методов, которые они не используют»** (*Clients should not be forced to depend upon interfaces that they do not use*).
Когда интерфейс разрастается до огромных размеров («толстый» интерфейс / Fat Interface), он объединяет в себе методы для совершенно разных потребителей.

```
    [ ТОЛСТЫЙ ИНТЕРФЕЙС (Нарушение ISP) ]
        +----------------------+
        |   ITransitTerminal   |
        +----------------------+
        | + read_rfid_card()   | ---> Нужен только ридеру
        | + print_paper_receipt()| ---> Нужен только принтеру
        | + sync_with_cloud()  | ---> Нужен только сети
        | + play_buzzer_beep() | ---> Нужен только звуку
        +----------------------+
```

Если у нас появится новый компактный валидатор без термопринтера чеков, разработчику придется писать в методе print_paper_receipt() заглушку pass или raise NotImplementedError.

Решение по ISP:

Дробим интерфейс на компактные, сфокусированные **ролевые протоколы**:

```
class CardReadable(Protocol):
    def read_card_id(self) -> str: ...

class ReceiptPrintable(Protocol):
    def print_receipt(self, text: str) -> None: ...

class SoundAlertable(Protocol):
    def play_success(self) -> None: ...
    def play_error(self) -> None: ...
```

---

3. Dependency Inversion Principle (DIP)

Формулировка принципа: 1. **Высокоуровневые модули не должны зависеть от низкоуровневых модулей. Оба должны зависеть от абстракций.** 2. **Абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций.**

```
[ Традиционная зависимость (Плохо) ]
Высокоуровневый модуль (PaymentProcessor) ---> Низкоуровневый модуль (PostgresDatab
ase)
(Бизнес-логика привязана к деталям хранения)

[ Инверсия зависимостей (Хорошо) ]
Высокоуровневый модуль (PaymentProcessor) ---> [ Абстракция: ITransactionRepository ]
                                                                    ^
```

Низкоуровневый модуль (PostgresDatabase) ----------|
(И бизнес-логика, и база данных зависят от единого контракта)

Паттерн Dependency Injection (DI)

Вместо того чтобы класс сам создавал свои зависимости через self.db = PostgresDatabase():

```
# ПЛОХО: Жесткая привязка
class TripValidator:
    def __init__(self):
        self._db = PostgresDatabase(host="10.0.0.1", port=5432) # Жесткая зависимость!
        self._buzzer = LinuxGpioBuzzer()                        # Тестировать невозможно!

# ОТЛИЧНО: Внедрение зависимостей через конструктор (Constructor Injection)
class TripValidator:
    def __init__(self, db: TransactionRepository, buzzer: AudioNotifier):
        self._db = db
        self._buzzer = buzzer
```

Теперь в тестах мы передаем легкие фиктивные объекты (InMemoryRepository, SilentMockBuzzer), а в боевом коде — реальные драйверы.

---

4. Сборка графа объектов: DI-контейнер

В больших приложениях ручное создание сотен объектов (A(B(C(D())), E(F()))) становится громоздким. Эту задачу берет на себя **IoC/DI-контейнер**:

```
# infrastructure/container.py
from typing import Dict, Any, Type, Callable


class SimpleDIContainer:
    """Простой легковесный DI-контейнер."""
    def __init__(self):
        self._services: Dict[Type, Any] = {}
        self._factories: Dict[Type, Callable] = {}

    def register_instance(self, interface: Type, instance: Any) -> None:
        self._services[interface] = instance

    def register_factory(self, interface: Type, factory: Callable) -> None:
        self._factories[interface] = factory

    def resolve(self, interface: Type) -> Any:
        if interface in self._services:
            return self._services[interface]
        if interface in self._factories:
            return self._factories[interface](self)
        raise KeyError(f"Сервис {interface.__name__} не зарегистрирован в контейнере.")
```

Подготовка к Midterm (Рубежный контроль 1)

На 5-й неделе студенты сдают Рубежный контроль 1. На экзамене проверяются: 1. Знание теории инкапсуляции, полиморфизма, композиции и всех 5 принципов SOLID. 2.

Литература к лекции

1. **Мартин Р.** Чистая архитектура. – СПб.: Питер, 2021. (Главы 9, 10, 11: LSP, ISP, DIP).
2. **Фаулер М.** Рефакторинг. – М.: Диалектика, 2020.
3. **Liskov B., Wing J.** A Behavioral Notion of Subtyping. ACM TOPLAS, 1994.